

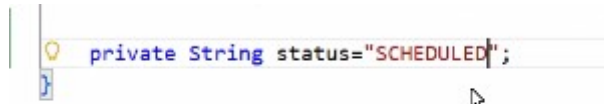
## Appunti del pomeriggio.

Soluzione al problema di sta mattina aggiungendo la possibilità di segnalare che la prenotazione è attiva, cioè che il cliente è arrivato.

Bisogna aggiungere al booking il momento del check in nel backend. Si comincia dal backend e gradualmente si risale al frontend.

Nel backend preparo uno stato che per semplicità sarà una stringa.

Private String status;

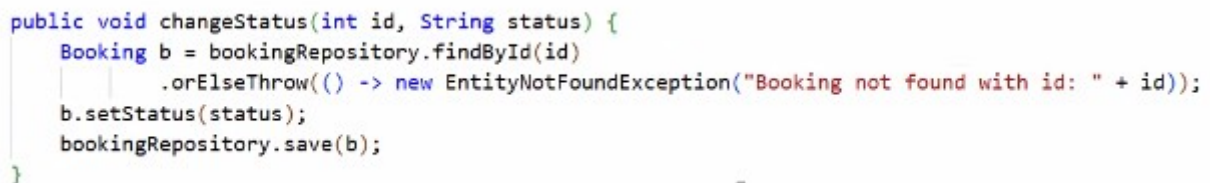


```
private String status = "SCHEDULED";
```

Aggiorno di conseguenza booking e booking dto.

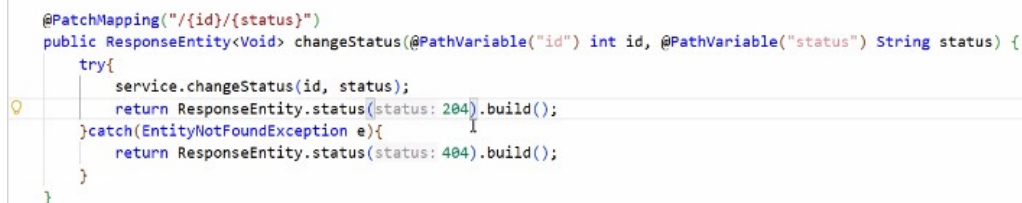
Nel DTO non ha un valore predefinito perché quando inserisco la prenotazione è SCHEDULED

Ora vado in BookingService, e faccio il metodo changeStatus



```
public void changeStatus(int id, String status) {  
    Booking b = bookingRepository.findById(id)  
        .orElseThrow(() -> new EntityNotFoundException("Booking not found with id: " + id));  
    b.setStatus(status);  
    bookingRepository.save(b);  
}
```

Fatto il service ci spostiamo nell'API e scriviamo il metodo corrispondente.

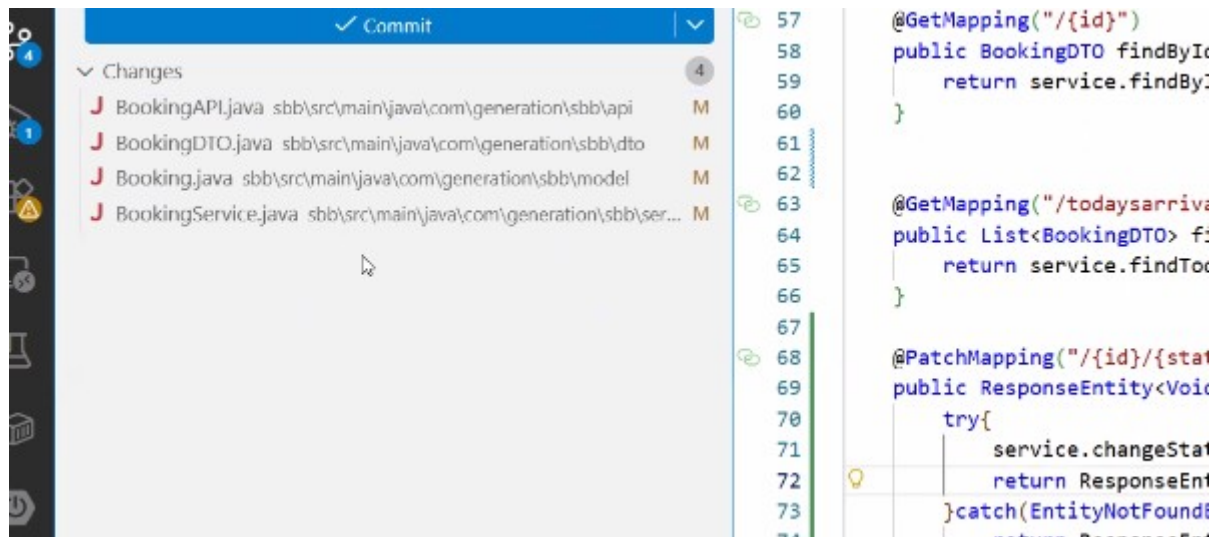


```
@PatchMapping("/{id}/{status}")  
public ResponseEntity<Void> changeStatus(@PathVariable("id") int id, @PathVariable("status") String status) {  
    try{  
        service.changeStatus(id, status);  
        return ResponseEntity.status(204).build();  
    } catch (EntityNotFoundException e){  
        return ResponseEntity.status(404).build();  
    }  
}
```

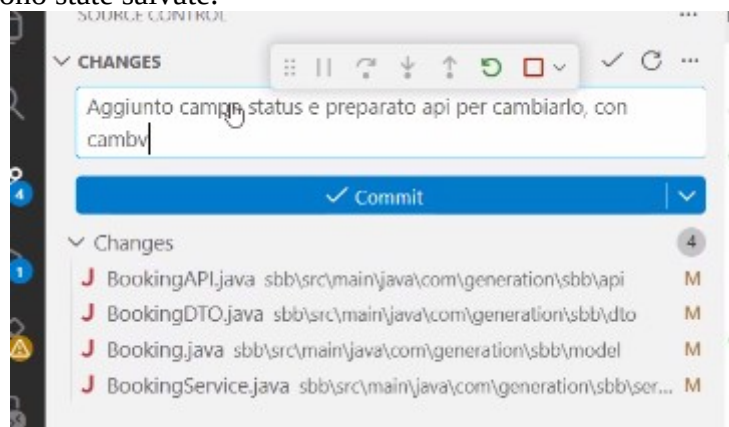
Provo se il mio backend funziona e avvio lo spring boot.

## GIT E SOURCE CONTROL

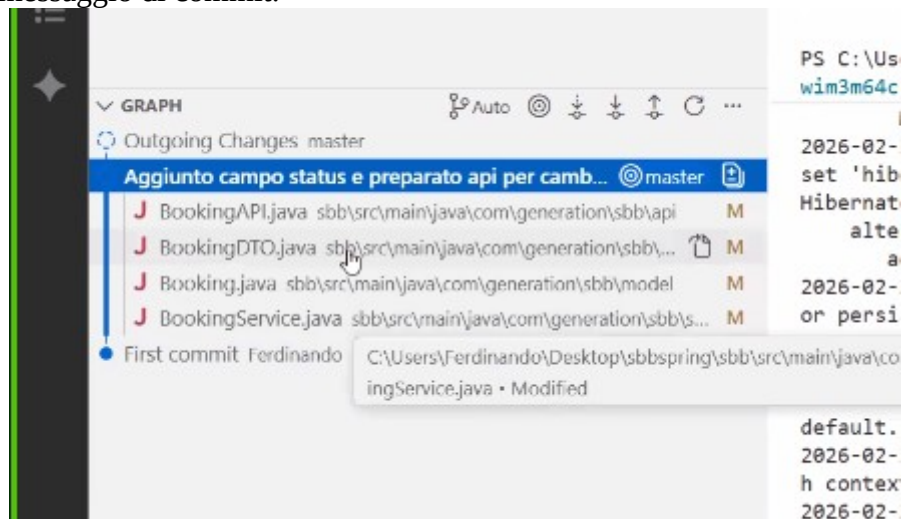
Quando abbiamo creato il progetto ho creato un repository locale, un archivio che memorizza tutte le versioni di un file. Il first commit è la prima versione. Quando si scrive un programma reale l'idea di avere una sola versione dei file è fantasia pura. Quando si fanno delle modifiche si distruggono pezzi del lavoro tuo o di qualcun'altro. Git è un sistema di versioning, vuol dire che quando io ho creato un repository, in questa cartella c'è una storia di tutti i file e posso tornare alle vecchie versioni.



Questi file fanno parte del mio progetto e sono stati modificati, git avvisa che nell'ultimo commit quelle versioni non sono state salvate.

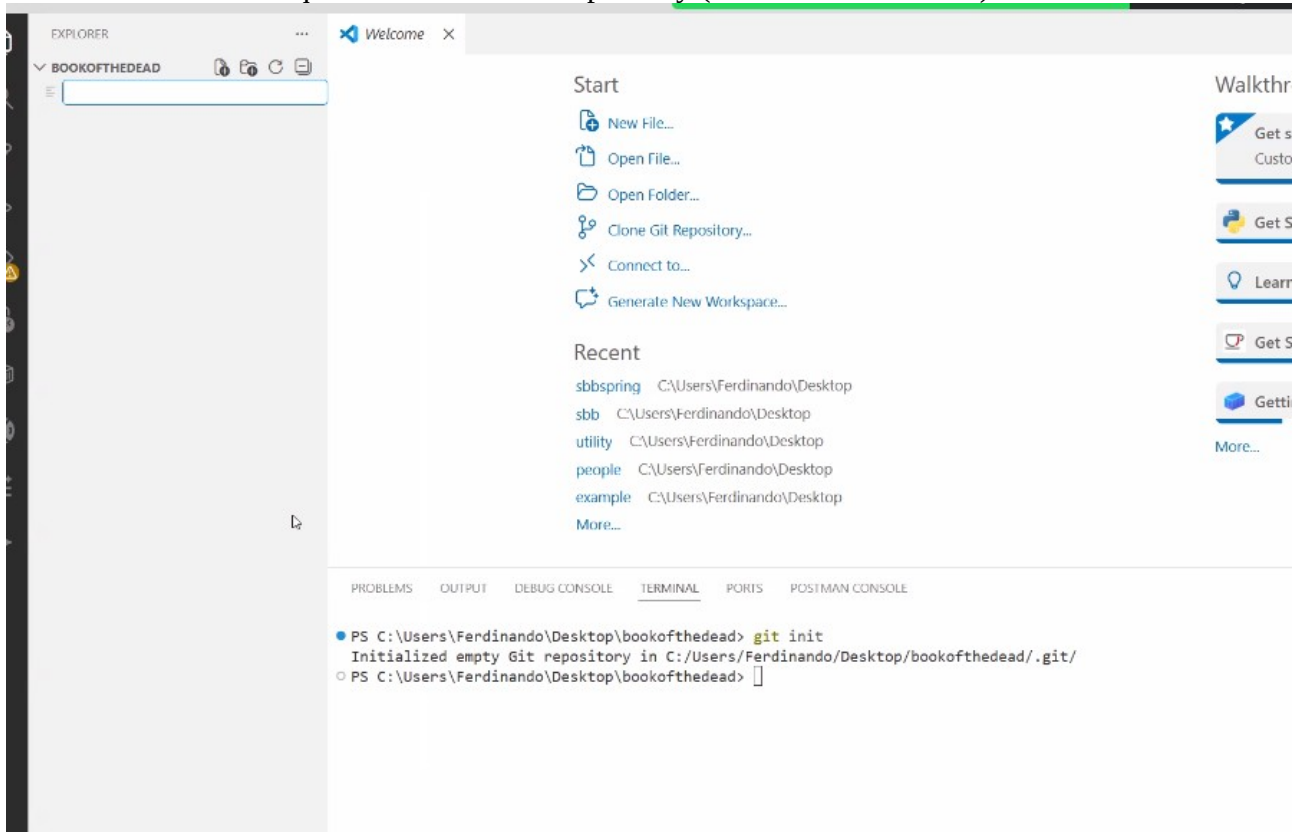


Si aggiunge il messaggio di commit.

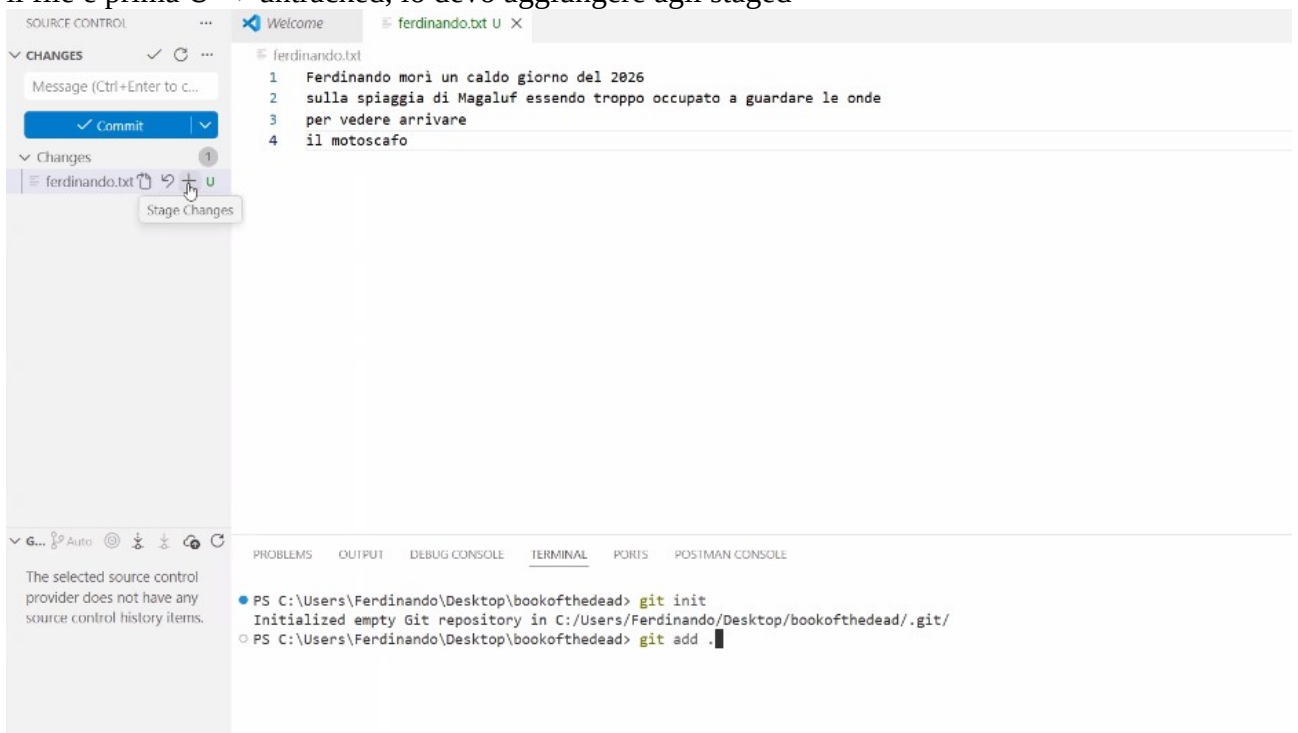


Io ho lavorato sul repository locale, dopo aver fatto le modifiche si fanno i commit per salvare tutto in locale e poi in remoto.

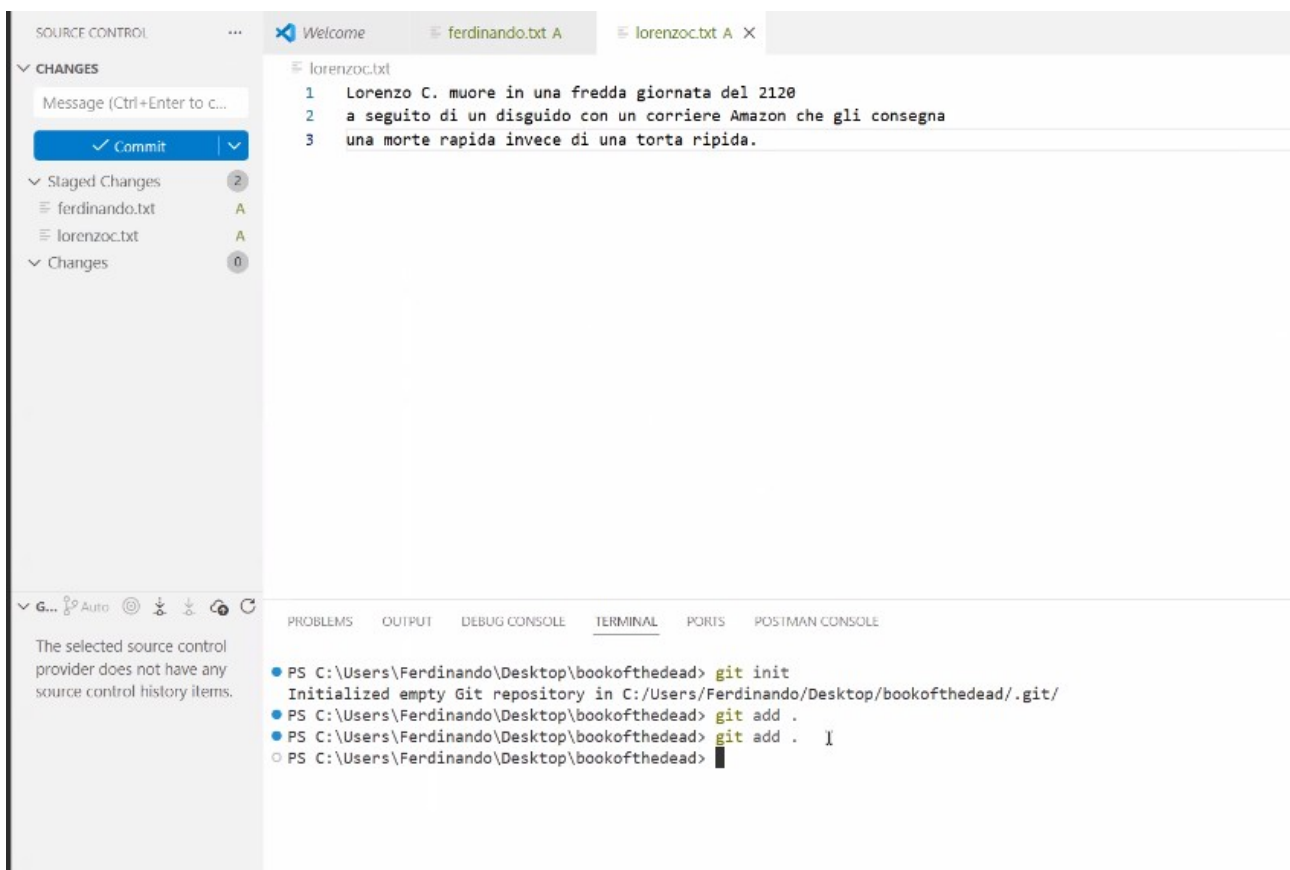
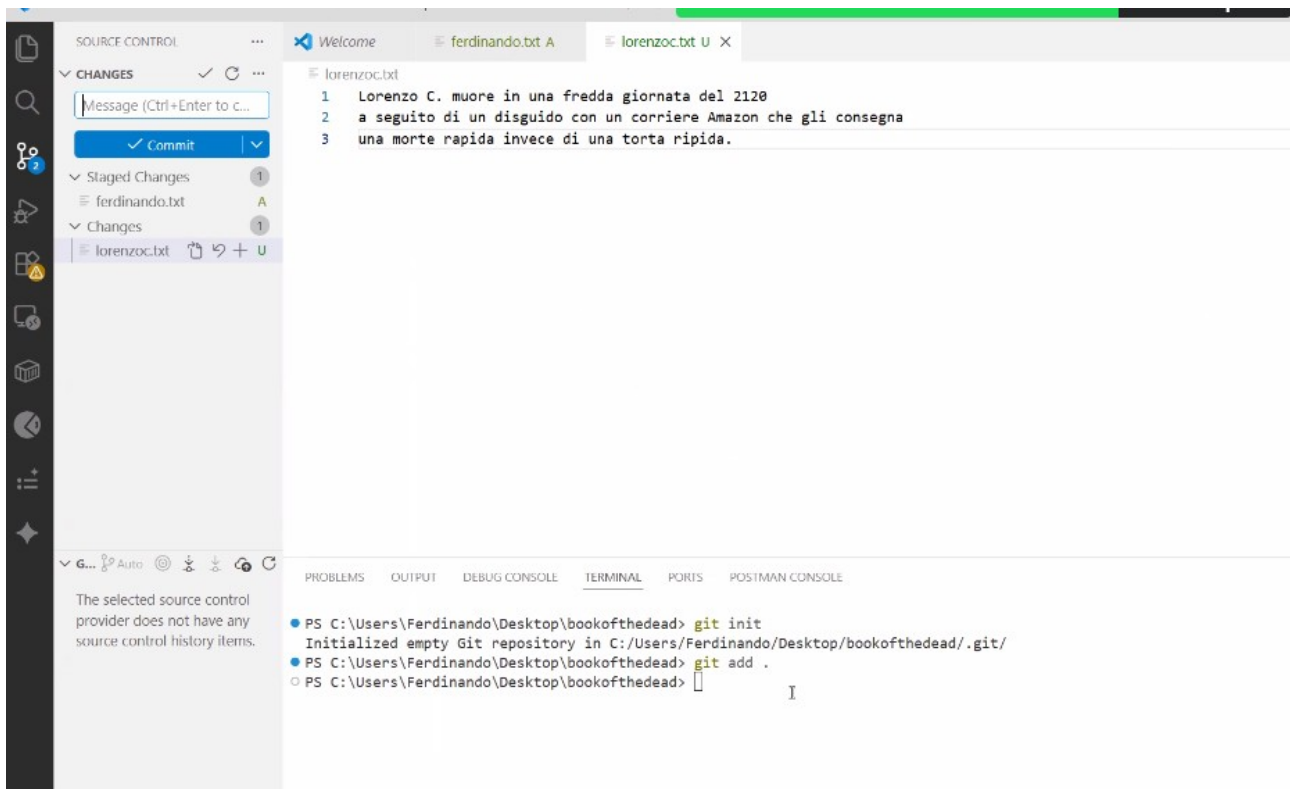
GIT INIT → dico che questa cartella è una repository (un archivio di codice).



il file è prima U → untracked, lo devo aggiungere agli staged

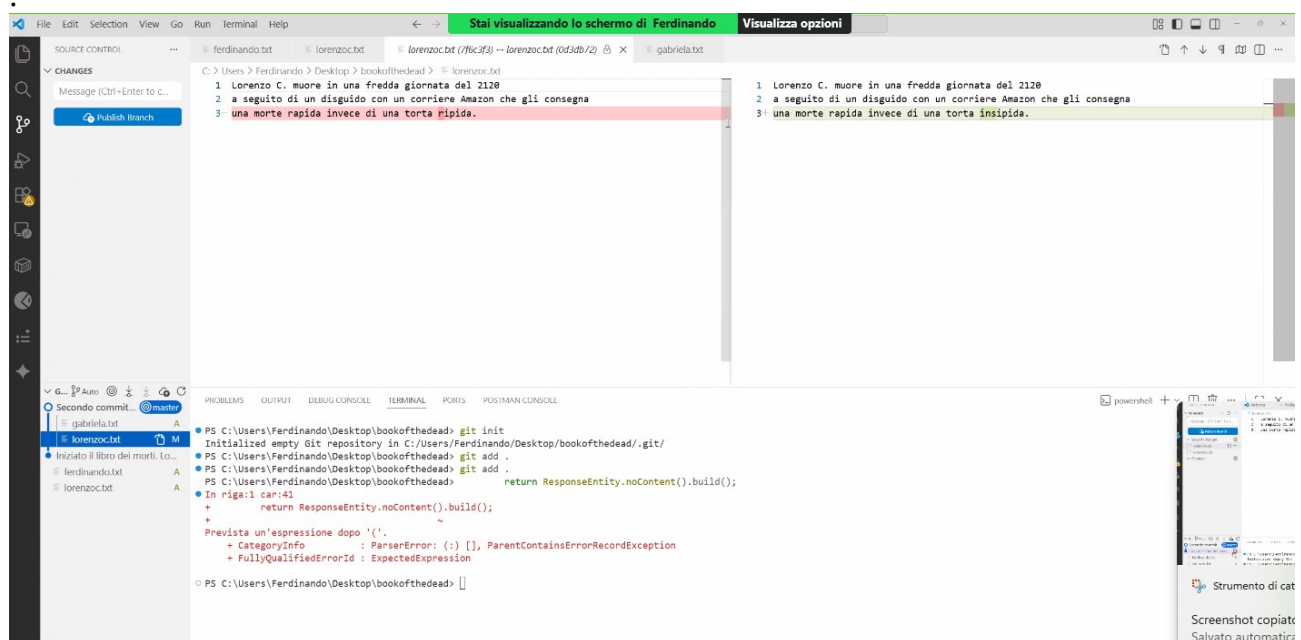
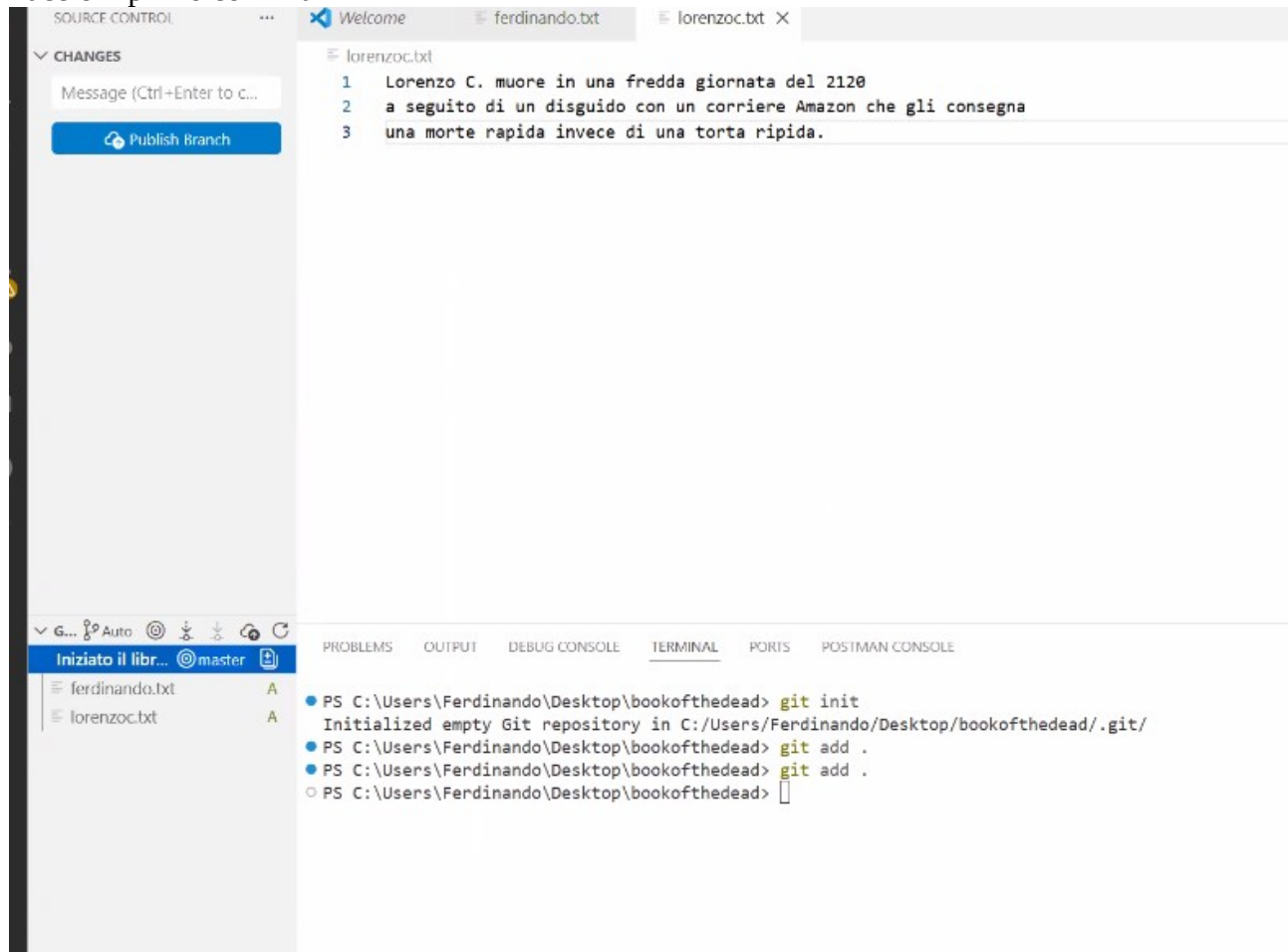


diventa A → index added



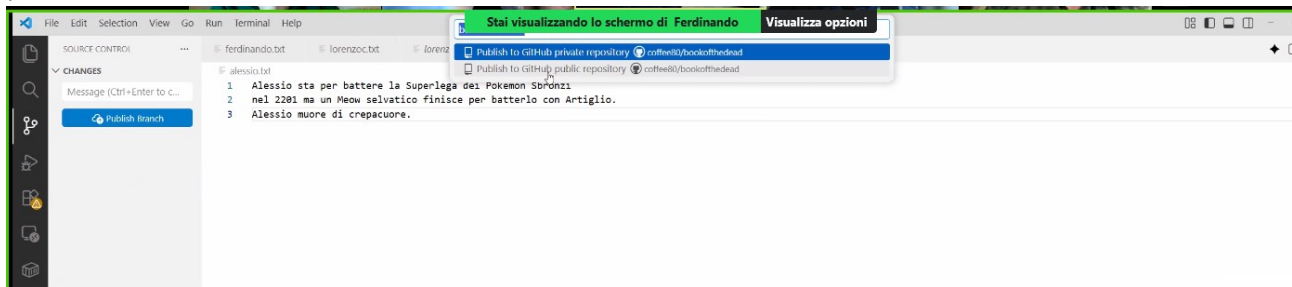
(per i progetti angular bisogna riscaricare le librerie perché sarebbero troppo pesanti per essere pushate sul remoto e non ha senso e si fa dinuovo npm install) .

## Faccio il primo commit



quando facciamo git abbiamo un archivio versionato in locale.  
Github è un repository remoto. Il repository remoto.





The screenshot shows an IDE with two Java files open. The left file is `BookingAPI.java` and the right file is `BookingAPI.java`. Both files contain a public class `BookingAPI` with methods `findById`, `findAll`, and `changeStatus`. The `BookingAPI.java` file on the right has a highlighted section of code.

```

// BookingAPI.java (Left)
22 public class BookingAPI {
55     public BookingDTO findById(@PathVariable int id) {
57     }
58
59     @GetMapping("/todaysarrivals/{hotelId}")
60     public List<BookingDTO> findAll(@PathVariable("hotelId") int hotelId) {
61         return service.findTodaysArrivalForHotel(hotelId);
62     }
63 }

// BookingAPI.java (Right)
25 public class BookingAPI {
60     public BookingDTO findById(@PathVariable int id) {
61     }
62
63     @GetMapping("/todaysarrivals/{hotelId}")
64     public List<BookingDTO> findAll(@PathVariable("hotelId") int hotelId) {
65         return service.findTodaysArrivalForHotel(hotelId);
66     }
67
68     @PatchMapping("/{id}/{status}")
69     public ResponseEntity<Void> changeStatus(@PathVariable("id") int id, @Pat
70     try{
71         service.changeStatus(id, status);
72         return ResponseEntity.status(204).build();
73     }catch(EntityNotFoundException e){
74         return ResponseEntity.status(404).build();
75     }
76 }
77
78
79

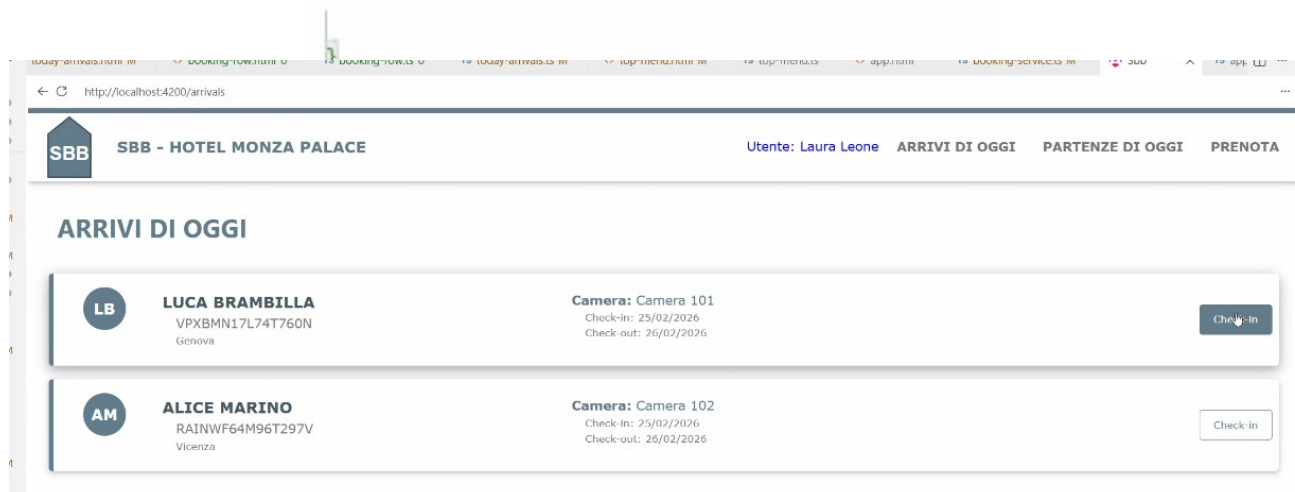
```

Ora sul Frontend ho bisogno di portare lo status.  
Prima cosa da fare andare in model e modificare il booking

```
19 export interface Booking{
20   id?:number;
21   guestId:number;
22   roomId:number;
23   guest:Guest;
24   room:Room;
25   from:Date;
26   to:Date;
27   notes:string;
28   status:string;
29   price:number;
30 }
31
32 export interface Hotel{
33   id:number;
```

Mi sposto in Booking Service e preparo un altro metodo, doCheckIn e gli arriva id di prenotazione.  
Il frontend chiama il backend.

```
public doCheckIn(id:number):Observable<Booking>{
  let url = this.apiUrl+"/"+id+"/executed";
  return this.http.patch<Booking>(url, null);
}
```



Quando clicchiamo su check in voglio chiamare il service, e il service chiamerà l'api.  
Booking Row ora ha bisogno del service e io glielo inietto.  
Vado su booking row.html e

```
<!-- Azioni -->
<div class="w3-col m2 s12 w3-right-align w3-padding-small">
  @if(booking().status!=='executed'){
    <button class="w3-button w3-small w3-round w3-border w3-border-blue-grey w3-text-blue-grey w3-hover-blue-grey w3-margin-top">
      Check-in
    </button>
  }
  @else{
  }
</div>

</div>

<!-- Azioni -->
<div class="w3-col m2 s12 w3-right-align w3-padding-small">
  @if(booking().status!=='executed'){
    <button (click)="doCheckIn()" class="w3-button w3-small w3-round w3-border w3-border-blue-grey w3-text-blue-grey w3-hover-blue-grey w3-margin-top">
      Check-in
    </button>
  }
</div>
```

a questo punto scrivo il metodo dentro bookingrow.

?? → **nullish coalescing operator**, operatore di coalescenza "nulla", ed è disponibile in TypeScript e JavaScript moderno.

```
doCheckIn():void{  
    let id:number = this.booking().id ?? 0;  
    this.bookingService.doCheckIn(id);  
}
```

Serve a dire: "Usa questo valore, ma se è `null` o `undefined`, usa quest'altro al suo posto".

```
16 |   styleUrls: ['./booking-row.css'],  
17 | })  
18 | export class BookingRow {  
19 |  
20 |   // ricevo un dato da graficare, come se fosse un parametro  
21 |   // come fosse Antani  
22 |  
23 |  
24 |   booking = model.required<Booking>();  
25 |  
26 |   bookingService = inject(BookingService);  
27 |  
28 |   doCheckIn():void{  
29 |  
30 |     let id:number = this.booking().id ?? 0;  
31 |  
32 |     this.bookingService.doCheckIn(id).subscribe(  
33 |     {  
34 |       next:()=>  
35 |       {  
36 |         this.booking.update(old=>{return {...old, status:"executed"}})  
37 |       },  
38 |       error:err=>console.log(err)  
39 |     });  
40 |   }  
41 |  
42 | }  
43 | }
```

SBB - HOTEL MONZA PALACE

Utente: Laura Leone ARRIVI DI OGGI PARTENZE DI OGGI PRENOTA

### ARRIVI DI OGGI

|    |                                                     |                                                                     |          |
|----|-----------------------------------------------------|---------------------------------------------------------------------|----------|
| LB | <b>LUCA BRAMBILLA</b><br>VPXBMN17L74T760N<br>Genova | Camera: Camera 101<br>Check-in: 25/02/2026<br>Check-out: 26/02/2026 | Check in |
| AM | <b>ALICE MARINO</b><br>RAINWF64M96T297V<br>Vicenza  | Camera: Camera 102<br>Check-in: 25/02/2026<br>Check-out: 26/02/2026 | Check in |

Analisi pagina Console Debugger Rete Editor stili Prestazioni Memoria Archiviazione Accessibilità Applicazione

Filtra messaggi

[vite] connecting...  
[vite] connected.  
Angular is running in development mode.  
Object { email: "laura.leone@fake.email.com", firstName: "Laura", hotel: {}, id: 2, lastName: "Leone", role: "RECEPTIONIST", username: "laura.leone" }  
Caricamento per hotel: 1  
http://localhost:8080/sbb/api/bookings/todayarrivals/1

client:733:9  
client:827:12  
debug\_node-chunk.js:10555:13  
user-service.ts:30:37  
today-arrivals.ts:50:25  
booking-service.ts:19:17

la prenotazione di Luca Brambilla è il componente booking-row che usa boooling service.



Richiesta:

Se una prenotazione è SCHEDULED → due pulsanti (CHECK-IN / CANCELED)

Se la prenotazione è EXECUTED → pulsante per indicare che abbiamo fatto le pulizie, un campo booleano da false a true. che deve comparire solo se la prenotazione è in giornata di chiusura.

Dopo la giornata del check out → abbiamo un campo cleaned (disponibile dopo il giorno del check out)

I pulsanti che vedo dipendono dallo stato della prenotazione.